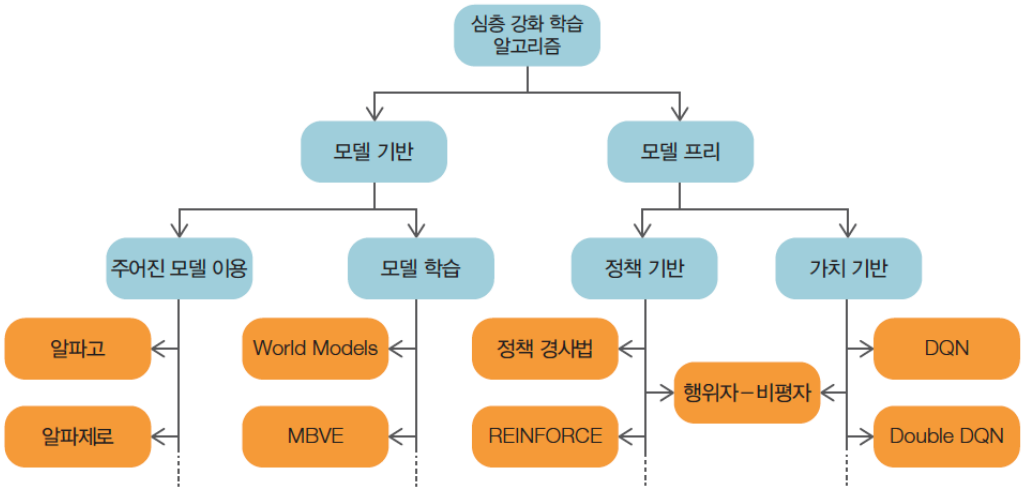
Advanced Methods

Prof. Tae-Hyoung Park
Dept. of Intelligent Systems & Robotics, CBNU

Deep Reinforcement Learning

- Model-Based Methods
  - state-transition function $p(s'|s,a)$, reward function $r(s,a,s')$ 사용
  - 환경 모델과 실제 환경 사이에 편향 발생 시 성능 저하 가능
    - Environment model 이 주어짐: AlphaGo, AlphaZero
    - Environment model 을 학습: World Models, Model-based value estimation
- Model-Free Methods
  - Policy Gradient 계열: Policy gradient, REINFORCE 등
  - Value 계열: DQN, Double DQN, Rainbow 등
  - Actor-Critic 계열: Actor-critic, A2C, A3C, DDPG, TRPO, PPO, SAC 등



Policy Gradient 계열

- Policy Gradient

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \Phi_t \nabla_\theta \log \pi_\theta(A_t|S_t)\right]$$

$\Phi_t = G(\tau)$: Simple policy gradient

$\Phi_t = G_t$: REINFORCE

$\Phi_t = G_t - V_w(S_t)$: Actor-Critic

$\Phi_t = R_t + \gamma V_w(S_{t+1}) - V_w(S_t)$

$\approx Q(S_t, A_t) - V(S_t) = A(S_t, A_t)$: Advantage Actor-Critic (A2C)

Actor-Critic 계열

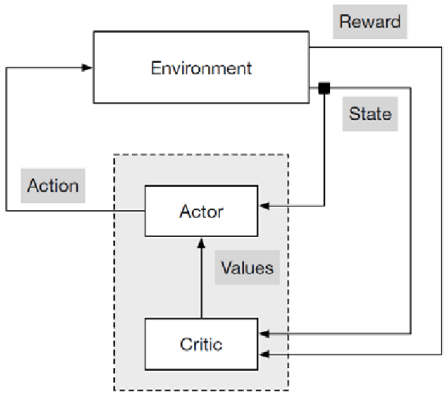
- A3C – Multiple Agents

  Asynchronous Advantage Actor-Critic, 2018, DeepMind

  – Actor-Critic architecture
    - Actor = policy network, Critic = value network

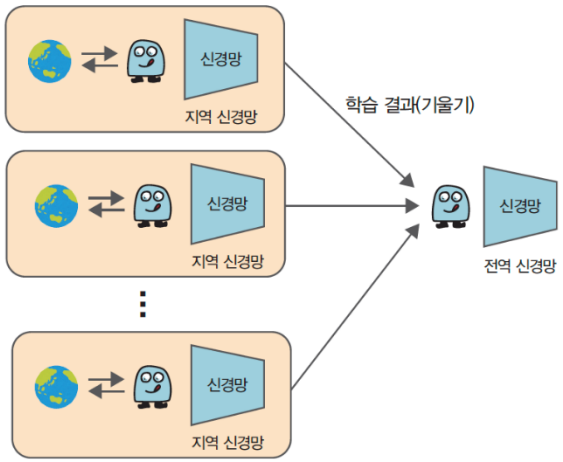
  – Multiple agents 가 병렬로 행동하며 비동기적으로 매개변수 update
    - Local network: 각자의 환경에서 독립적으로 플레이하며 학습 수행 (학습결과=기울기)
    - Global network: 여러 local network 에서 보내온 기울기를 이용해 비동기적으로 가중치 매개변수 update

(Actor-Critic)



Actor: policy network
Critic: value network

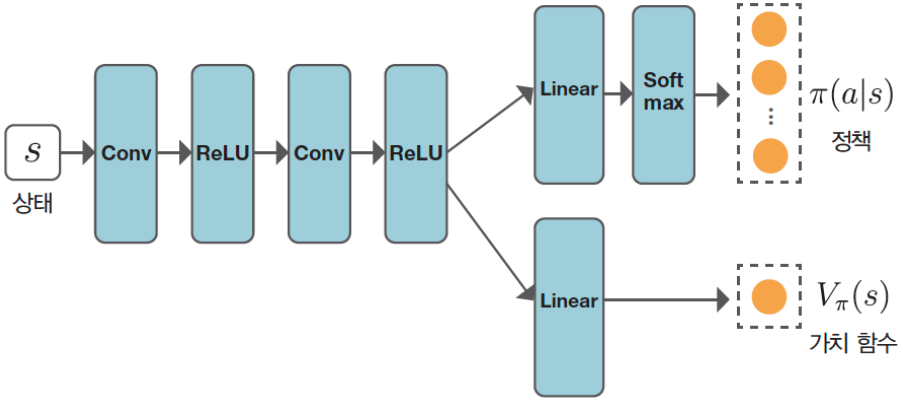
(A3C)



Actor-Critic 계열

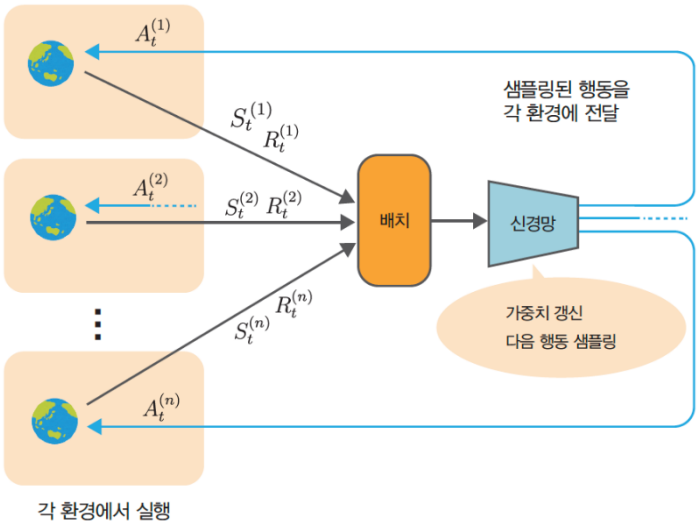
- A3C (계속)
  - 특징
    - 여러 agent 에 대하여 병렬 처리 가능
    - Agent 들이 독립적으로 행동하기 때문에 다양한 데이터 획득 가능 => 데이터의 상관관계 약화 => 안정적인 학습 가능
    - Actor (policy network) - Critic (value network) 의 신경망 가중치 공유 가능

(A3C 신경망 구조)



Actor-Critic 계열

- A2C (Advantage Actor-Critic) – Multiple Agents
  - Synchronous version of A3C
  - 각각의 환경에서 에이전트가 독립적으로 행동: $A_t^{(1)}, \cdots, A_t^{(n)}$
  - 시간 t 에서의 상태 ($S_t^{(1)}, \cdots, S_t^{(n)}$), ($R_t^{(1)}, \cdots, S_t^{(n)}$) 를 batch로 묶어 하나의 신경망 학습
  - 신경망의 출력 (policy 확률분포) 에서 다음 action 을 sampling 하여 각 환경에 전달: $A_{t+1}^{(1)}, A_{t+1}^{(2)}, \cdots, A_{t+1}^{(n)}$



	A3C	A2C
agents	N 개	N 개
network	N 개	1 개
hardware	Better for CPU	Better for GPU
stability	high	High, but needs careful tuning
Training speed	Slower per step	Faster per step

Actor-Critic 계열

- PPO (Proximal Policy Optimization)
  - Schlman et.al. (2017)
  - A2C 기반 framework
  - Clipped objective function 도입

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]$$

Where:

- $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the **probability ratio** between new and old policies.
- $\hat{A}_t$ is the **advantage estimate** at timestep t.
- ϵ is a small number (typically 0.1 to 0.3) that determines how much deviation is allowed.
- `clip(...)` limits $r_t(\theta)$ to the range $[1-\epsilon, 1+\epsilon]$.

  - New policy 가 급변하는 것을 방지 (“proximal”) => 학습 안정성 향상

Actor-Critic 계열

- DDPG

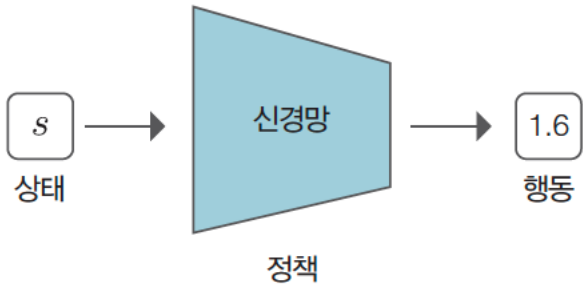
  Deep Deterministic Policy Gradient (Deep Mind, 2015)

  – Actor-Critic architecture
    - Actor: policy network $\mu_\theta(s)$
    - Critic: Q-value network $Q_\emptyset(s, a)$

  – Continuous action space
    - (ex) steering angle, throttle, ….

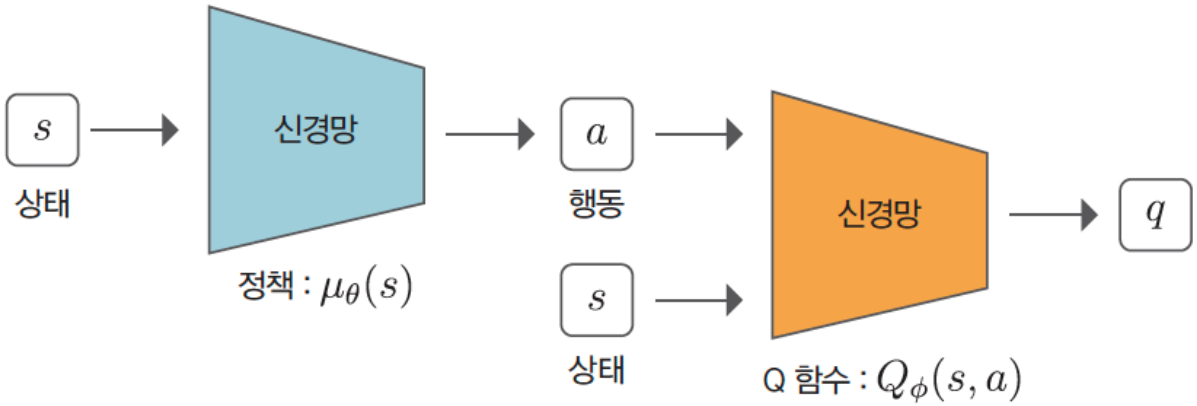
  – Deterministic Policy
    - Output = single deterministic action for each state

    (cf) A3C, PPO : output probabilistic distribution of action



Actor-Critic 계열

- DDPG (계속)
  - DQN based networks: $\mu_\theta(s)$, $Q_\phi(s,a)$
    - Replay buffer
    - Target networks
  - Exploration noise
    - Adds noise to actions during training



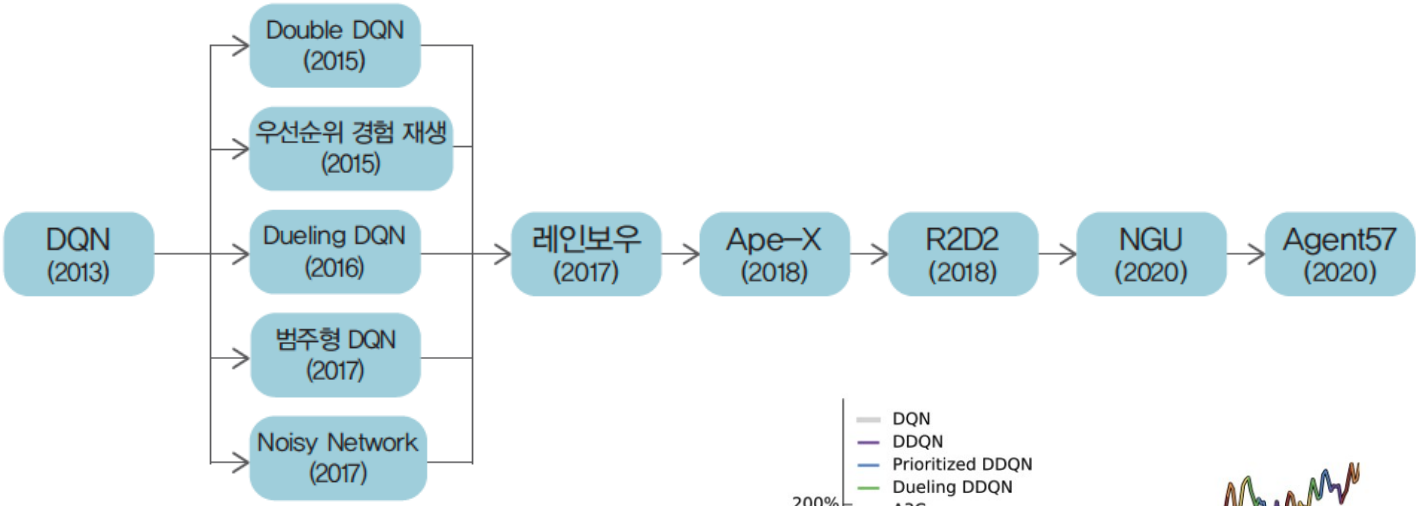
Actor-Critic 계열

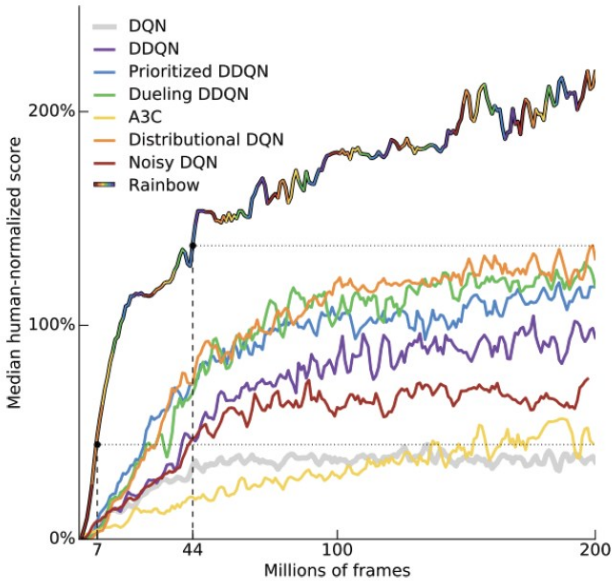
- SAC (Soft Actor-Critic)
  - Haarnoja et.al., 2018
  - Off policy : replay buffer 사용 => 데이터 재사용 가능 sample efficient
  - Entropy 기법 적용 => exploration 활성화
  - Soft policy evaluation (Critic): 두개의 Q 함수 사용 (double Q)
  - Soft actor evaluation (Actor)

  ⇒ 최적의 policy 를 학습하면서 충분한 탐험 (exploration) 을 유지하기 위해 "reward + entropy" 를 함께 최대화 시키는 actor-critic 알고리즘

DQN 계열

- DQN 기반 알고리즘





요약

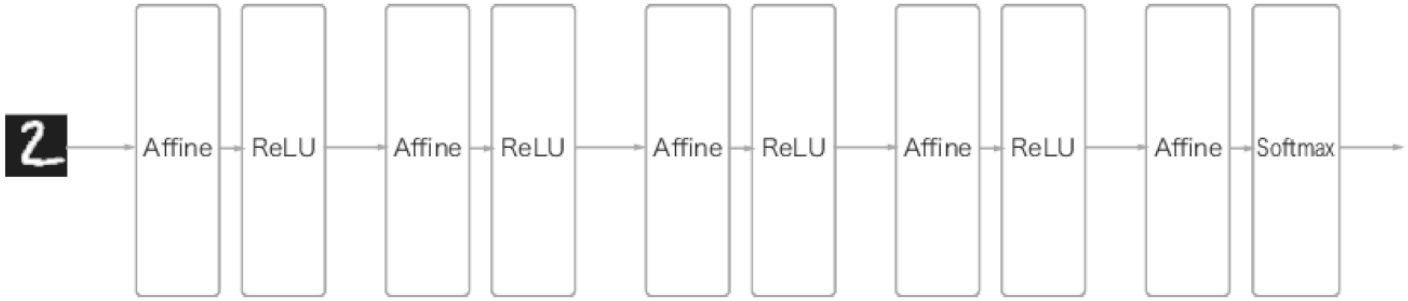
항목	DQN	A2C	PPO	SAC
전체 분류	Value-based	Policy-based (Actor-Critic)	Policy-based (Actor-Critic)	Off-policy Actor-Critic
정책 타입	Deterministic	Stochastic	Stochastic	Stochastic
행동 공간	이산 (Discrete)	이산 또는 연속	이산 또는 연속	연속 (Continuous)
Off-policy / On-policy	Off-policy	On-policy	On-policy	Off-policy
탐험 전략	ε-greedy	내재적 (확률적 정책)	내재적 + 클리핑	확률적 + 엔트로피 최대화
샘플 효율성	중간	낮음	중간	**높음**
안정성	중간	낮음	**높음**	**높음**
복잡성	낮음	낮음	중간	**높음**
사용 분야	이산 환경 (게임 등)	일반 강화학습	범용 (로봇, 게임)	**연속 제어 (로봇, 제어)**

Convolution Neural Network

Prof. Tae-Hyoung Park
Dept. of Intelligent Systems & Robotics, CBNU

CNN Structure

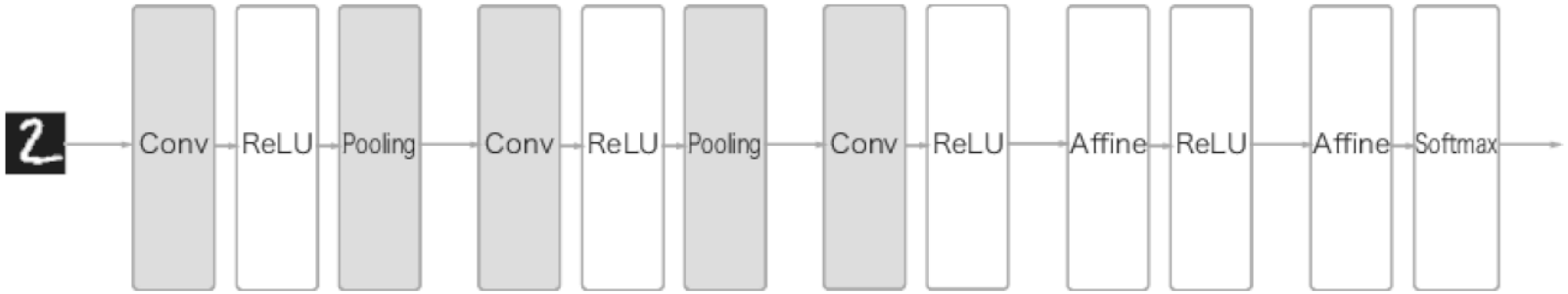
- Deep neural network (fully connected network)



영상 데이터 -> 1차원 벡터 : 입력 영상의 '형상' 무시

(ex) 32 pixel x 32 pixel x 3 channel image -> 3072 x 1 vector

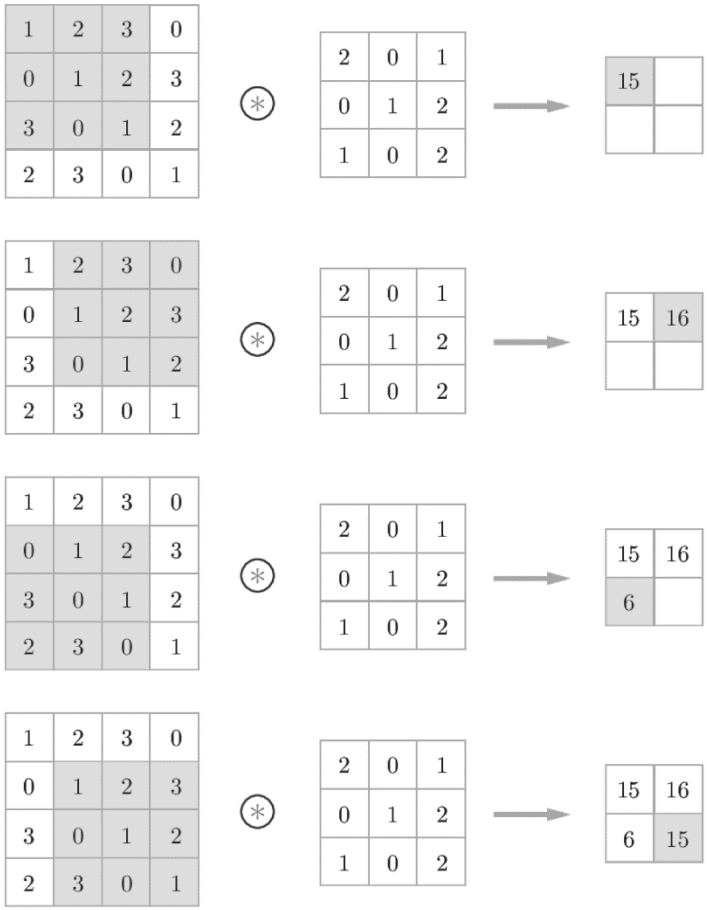
- Convolution neural network

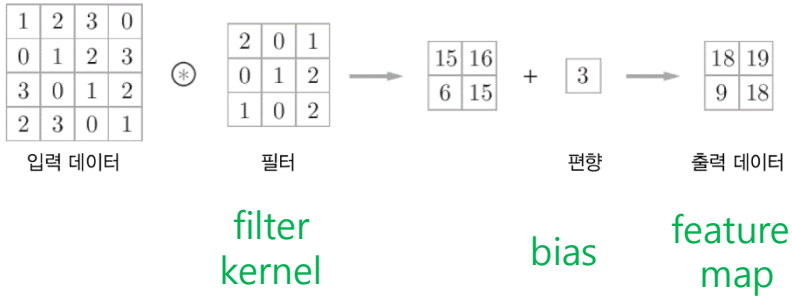


영상 데이터 -> 3차원 벡터 (가로,세로,채널) : 입력 영상의 '형상' 유지

Convolution Layer

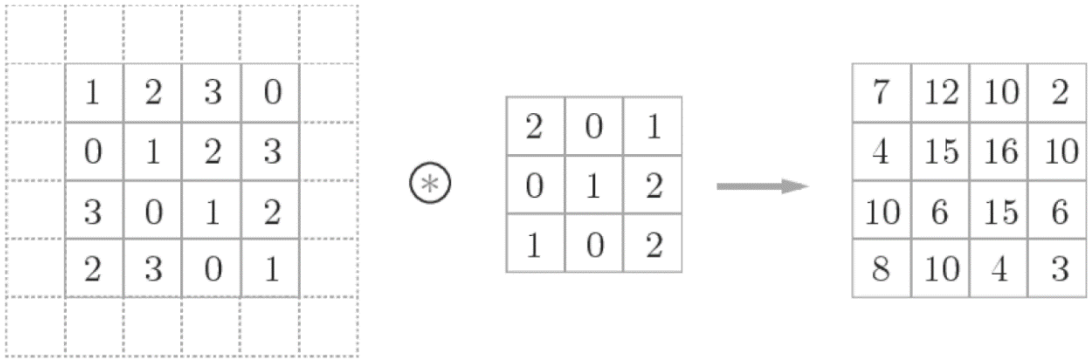
- Convolution 연산
  - 이웃 pixel 들의 공간적 특성값 (spatial feature)





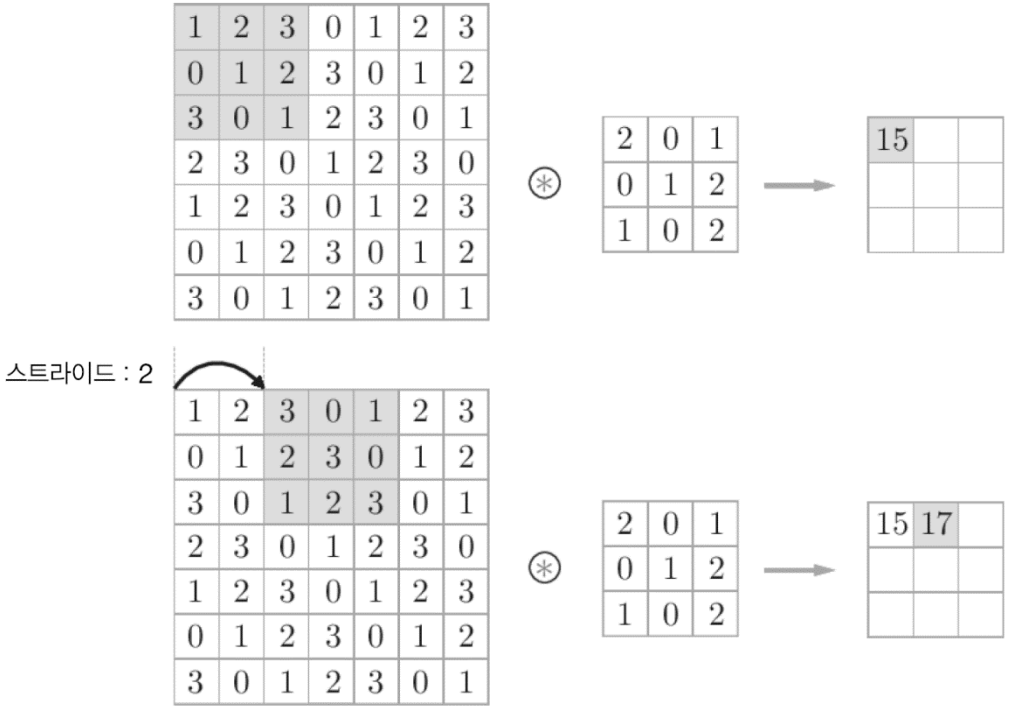
Convolution Layer

- Padding
  - 영상 모서리의 convolution 연산을 위하여, 입력 데이터 주변을 특정값으로 채움



Convolution Layer

- Stride
  - Filter 를 적용하는 위치 간격



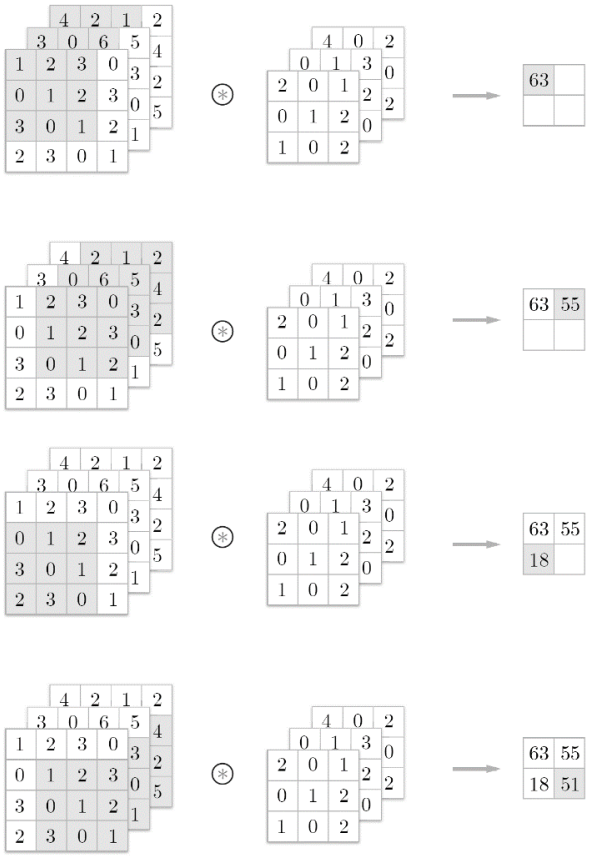
$$OH = \frac{H + 2P - FH}{S} + 1$$

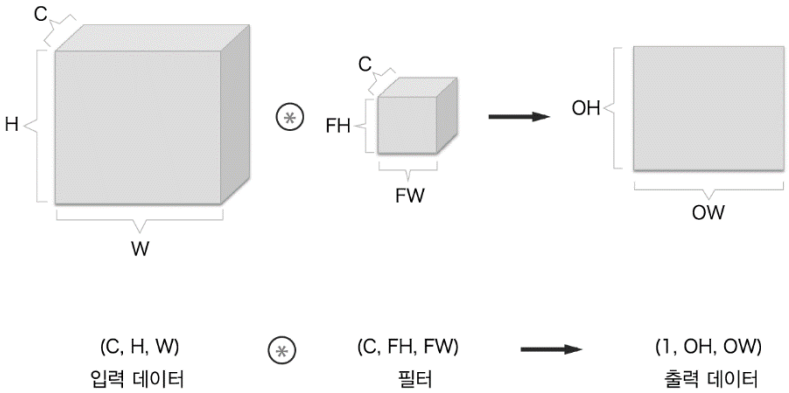
$$OW = \frac{W + 2P - FW}{S} + 1$$

입력크기: (H,W), Filter 크기: (FH, FW), 출력크기: (OH, OW)
Padding: P, Stride: S

Convolution Layer

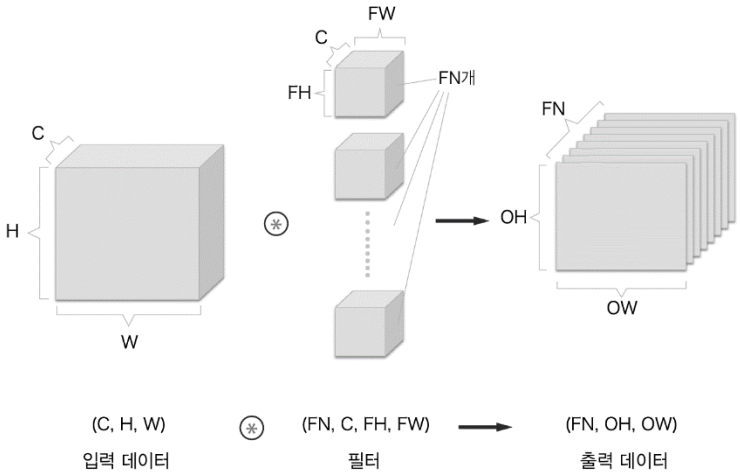
- Color image convolution
  - Convolution for 3 channels (R,G,B)
  - 3D data convolution





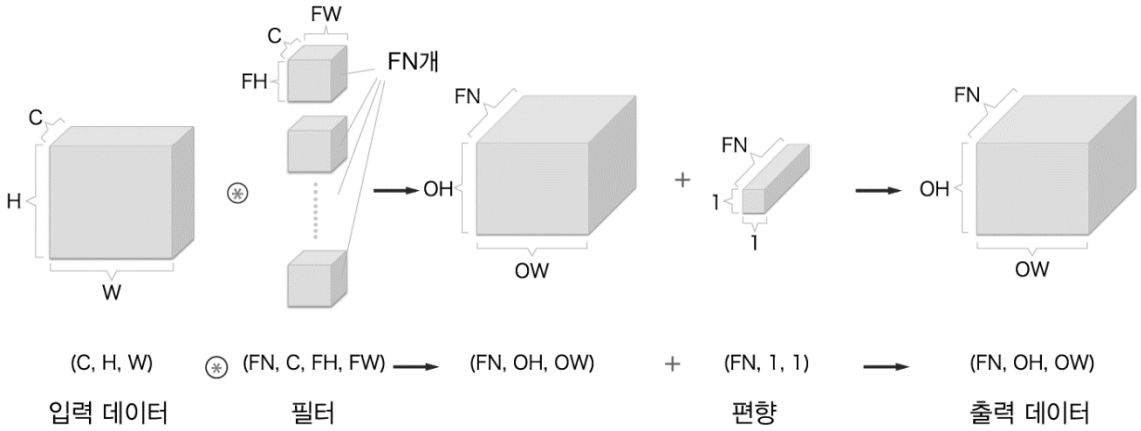
Convolution Layer

- 여러 개의 filter 사용



FN: 필터 갯수

- Bias 추가



Convolution Layer

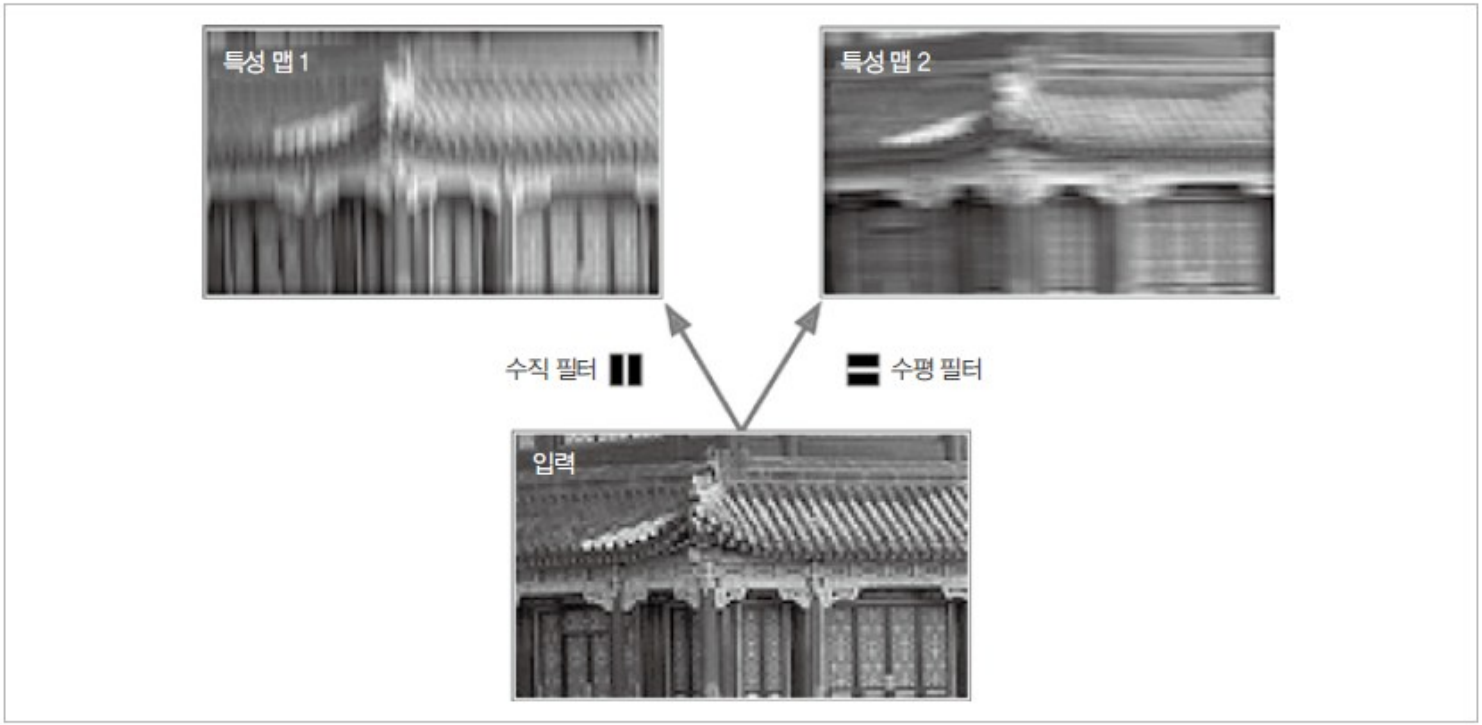
- Hyperparameters
  - Number of filters **K**
  - The filter size **F**
  - The stride **S**
  - The zero padding **P**

- 학습 parameter

  F^2CK and K biases (C: number of channels)

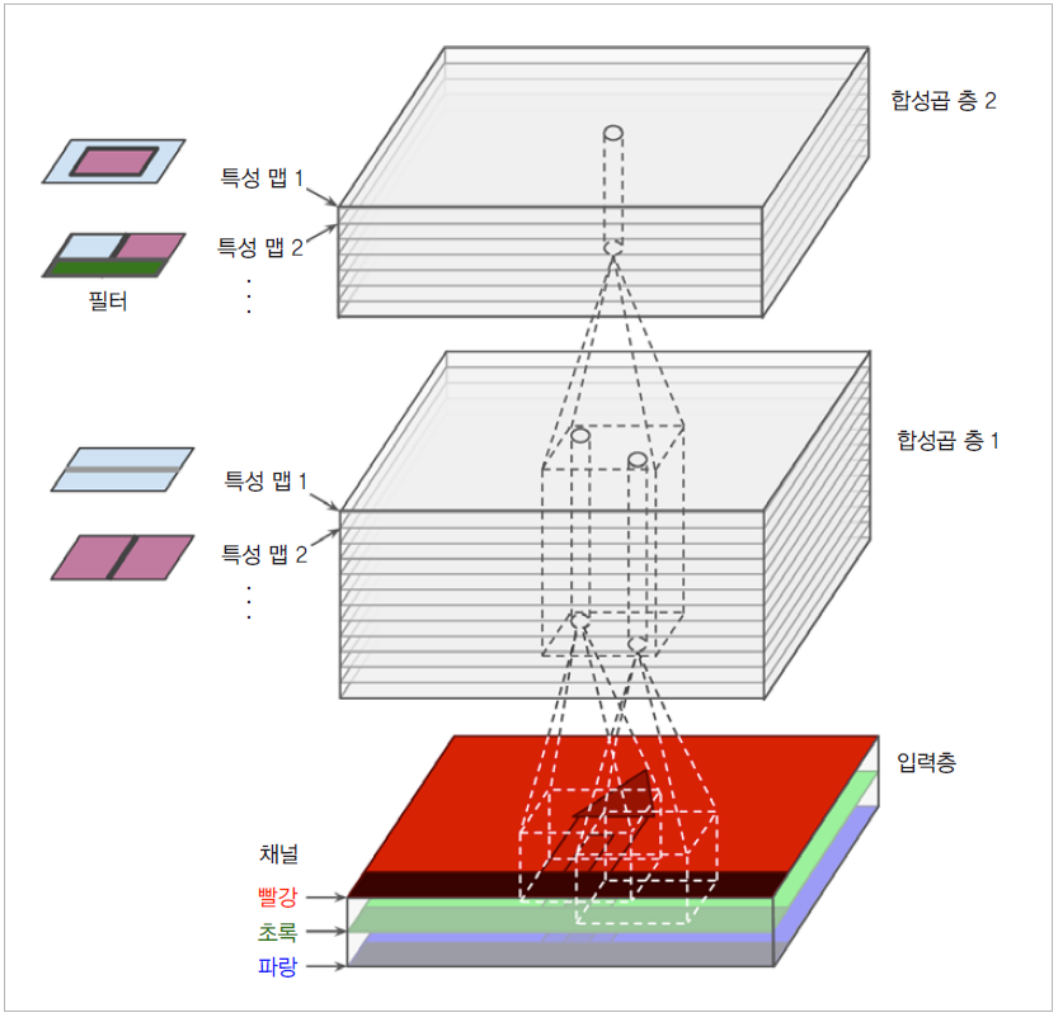
Convolution Layer

- Filter (kernel) & feature map



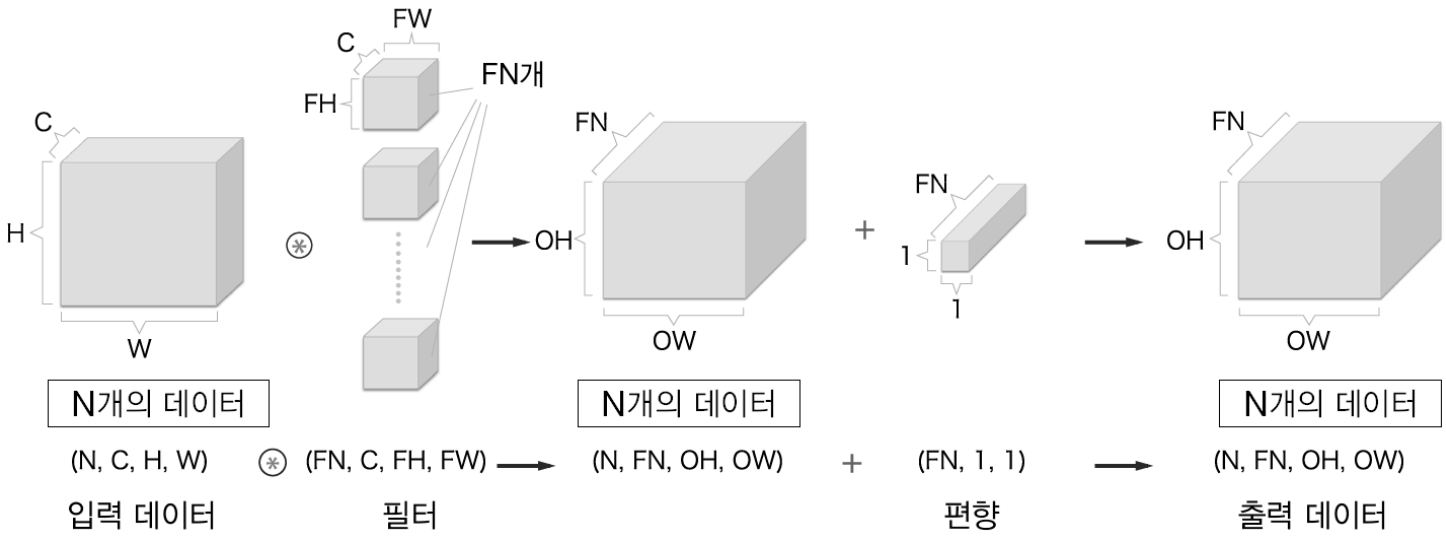
Convolution Layer

- Filter & Feature map



Convolution Layer

- Batch 처리



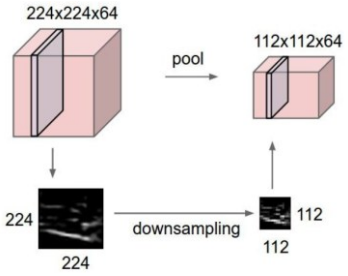
N: batch 데이터 수

- 4차원 데이터 (Tensor) 의 흐름
- 연산내용: 행렬 곱셈 & 덧셈
- Convolution 연산으로 local feature 를 추출하여 feature map 출력

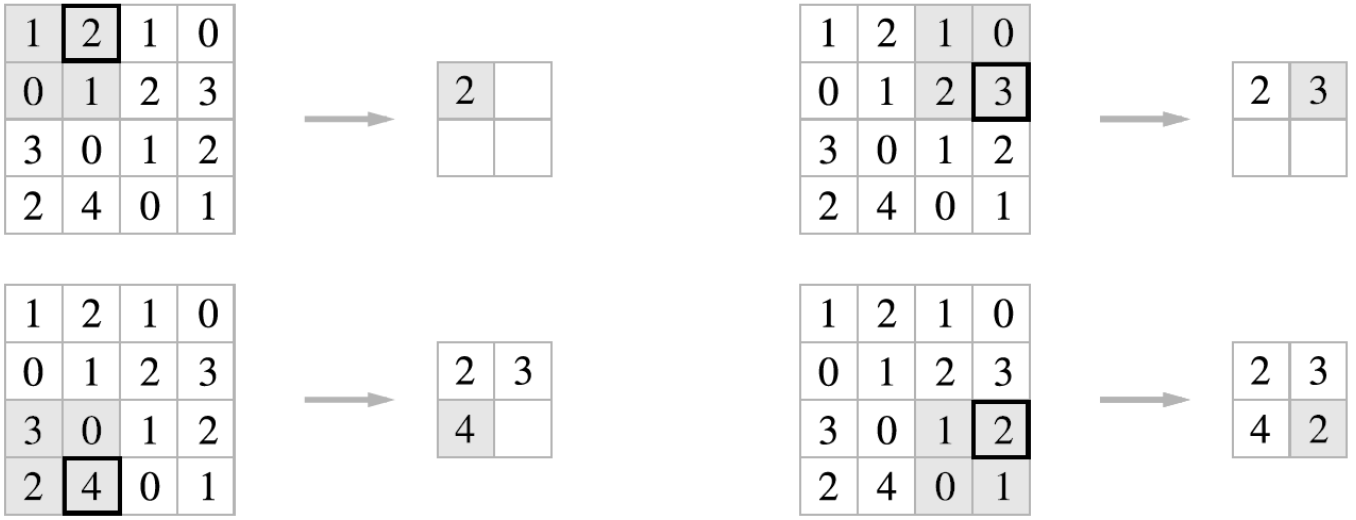
Pooling Layer

- Pooling layer
  - 가로 세로 방향의 공간을 줄이는 연산
  - 종류: Max pooling, average pooling 등
  - 학습 대상이 되는 parameter 가 없음

pooling: 취합/선별



Max-pooling



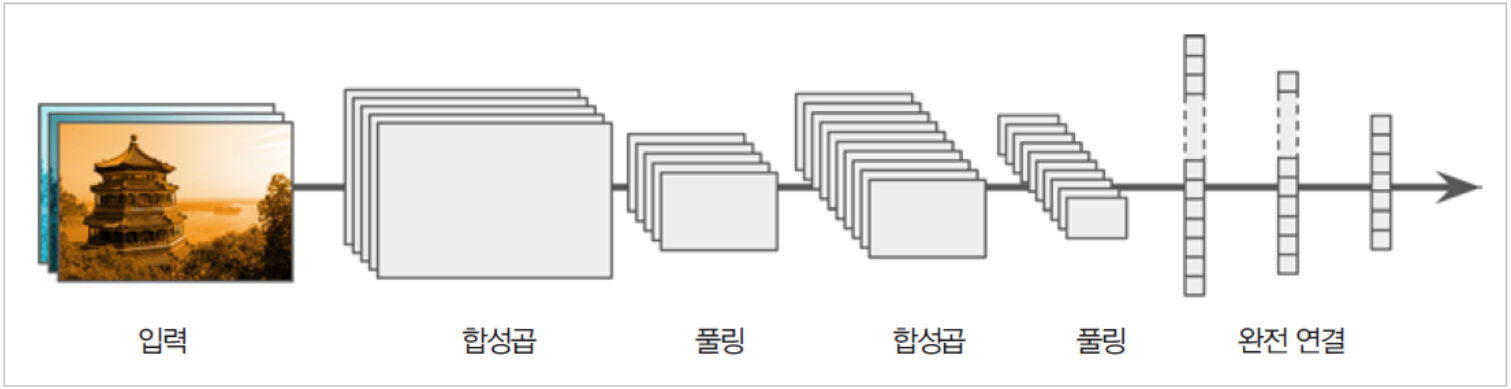
Pooling Layer

- Hyperparameter
  - Spatial extent **F**
  - Stride **S**

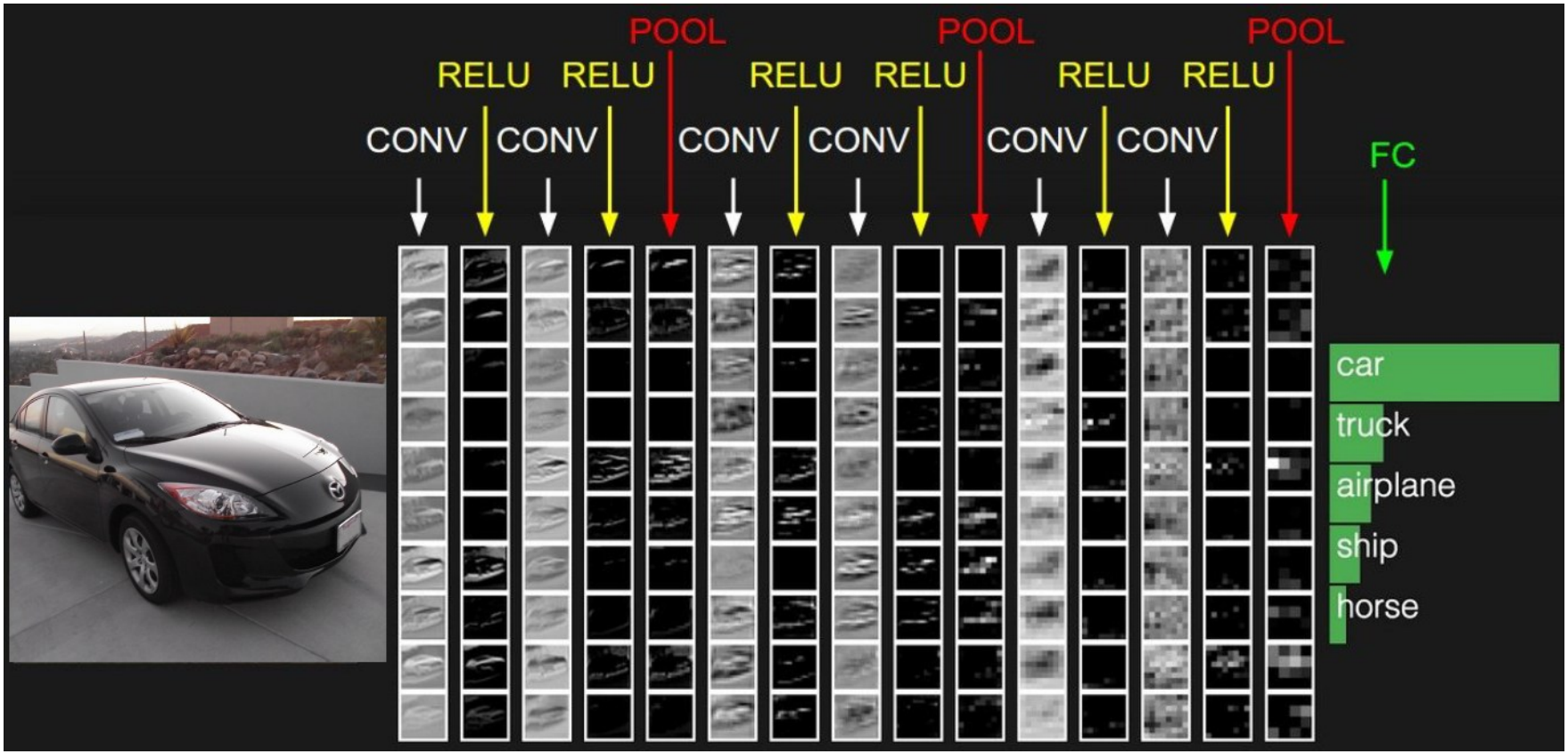
- 학습 parameter
  - none

기본 CNN 구조

- Typical CNN
  - Convolution layer + Pooling layer + Fully connected layer

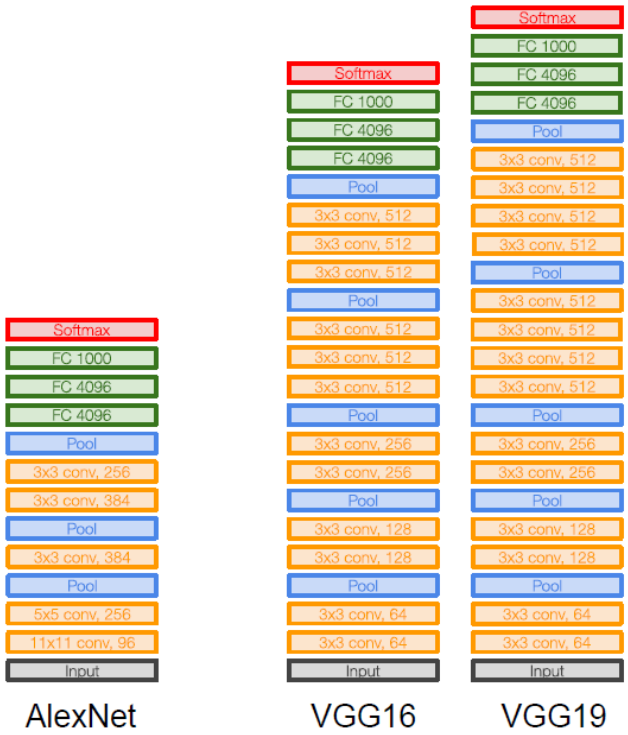


기본 CNN 구조


POOL
POOL
POOL
RELU
RELU
RELU
RELU
RELU
RELU
CONV
CONV
CONV
CONV
CONV
CONV
FC
car
truck
airplane
ship
horse

CNN 구조

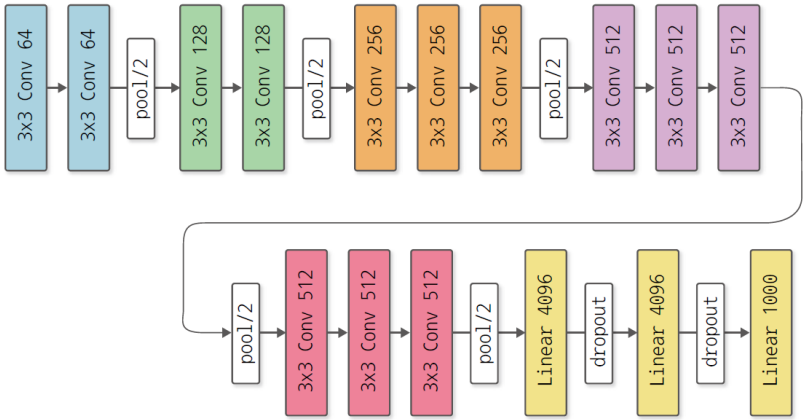
- VGGNet
  - Simonyan & Zisserman, 2014
  - Alex net 대비 Conv. Kernel size 를 줄이고 layer 를 늘림
    - 여러 개의 3x3 conv. 는 1개의 7x7 conv. 와 effective receptive field 가 같음
    - Deeper, more non-linearities



VGG16 구현

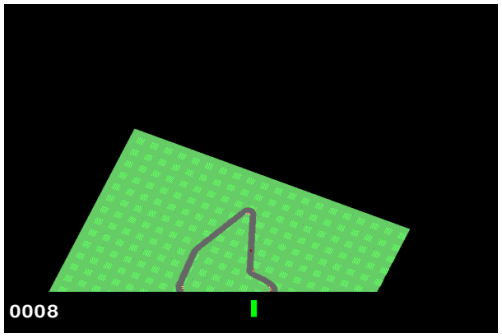
DeZero₩models.py

- Model




```python
class VGG16(Model):
    WEIGHTS_PATH = 'https://github.com/koki0702/dezero-models/releases/download/v0.1/vgg16.npz'

    def __init__(self, pretrained=False):
        super().__init__()
        self.conv1_1 = L.Conv2d(64, kernel_size=3, stride=1, pad=1)
        self.conv1_2 = L.Conv2d(64, kernel_size=3, stride=1, pad=1)
        self.conv2_1 = L.Conv2d(128, kernel_size=3, stride=1, pad=1)
        self.conv2_2 = L.Conv2d(128, kernel_size=3, stride=1, pad=1)
        self.conv3_1 = L.Conv2d(256, kernel_size=3, stride=1, pad=1)
        self.conv3_2 = L.Conv2d(256, kernel_size=3, stride=1, pad=1)
        self.conv3_3 = L.Conv2d(256, kernel_size=3, stride=1, pad=1)
        self.conv4_1 = L.Conv2d(512, kernel_size=3, stride=1, pad=1)
        self.conv4_2 = L.Conv2d(512, kernel_size=3, stride=1, pad=1)
        self.conv4_3 = L.Conv2d(512, kernel_size=3, stride=1, pad=1)
        self.conv5_1 = L.Conv2d(512, kernel_size=3, stride=1, pad=1)
        self.conv5_2 = L.Conv2d(512, kernel_size=3, stride=1, pad=1)
        self.conv5_3 = L.Conv2d(512, kernel_size=3, stride=1, pad=1)
        self.fc6 = L.Linear(4096)
        self.fc7 = L.Linear(4096)
        self.fc8 = L.Linear(1000)

        if pretrained:
            weights_path = utils.get_file(VGG16.WEIGHTS_PATH)
            self.load_weights(weights_path)

    def forward(self, x):
        x = F.relu(self.conv1_1(x))
        x = F.relu(self.conv1_2(x))
        x = F.pooling(x, 2, 2)
        x = F.relu(self.conv2_1(x))
        x = F.relu(self.conv2_2(x))
        x = F.pooling(x, 2, 2)
        x = F.relu(self.conv3_1(x))
        x = F.relu(self.conv3_2(x))
        x = F.relu(self.conv3_3(x))
        x = F.pooling(x, 2, 2)
        x = F.relu(self.conv4_1(x))
        x = F.relu(self.conv4_2(x))
        x = F.relu(self.conv4_3(x))
        x = F.pooling(x, 2, 2)
        x = F.relu(self.conv5_1(x))
        x = F.relu(self.conv5_2(x))
        x = F.relu(self.conv5_3(x))
        x = F.pooling(x, 2, 2)
        x = F.reshape(x, (x.shape[0], -1))
        x = F.dropout(F.relu(self.fc6(x)))
        x = F.dropout(F.relu(self.fc7(x)))
        x = self.fc8(x)
        return x
```

Car Racing

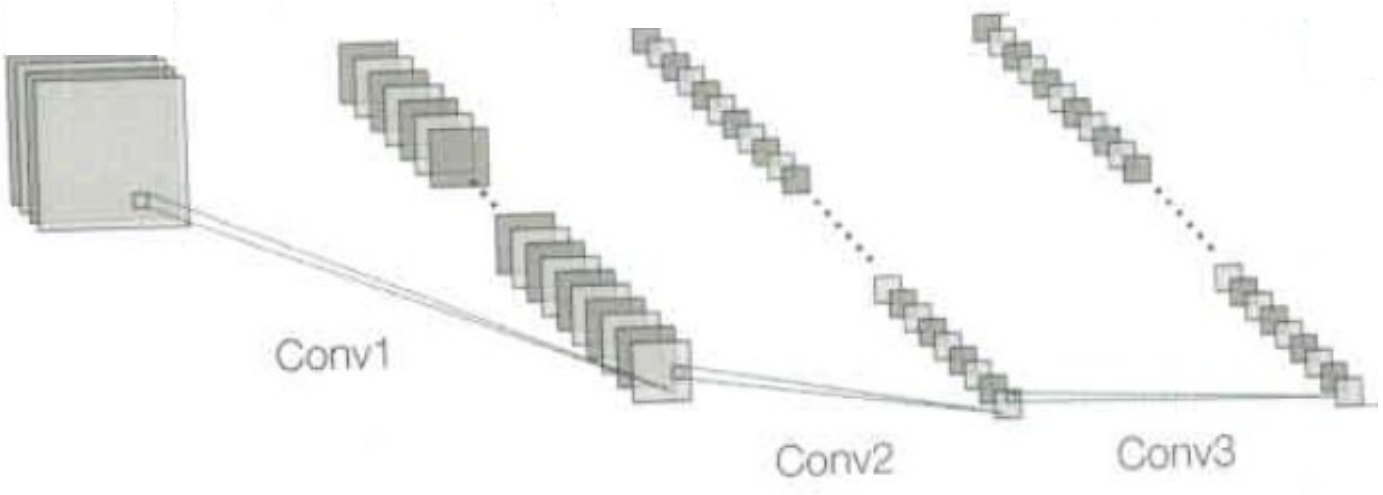


- Environment
  - State (observation)
    - 96 x 96 pixels (RGB)
  - Action
    - Discrete (5): do nothing, steer left, steer right, gas, brake
  - Rewards
    - -0.1 for every frame, +1000/N for evert track tile visited
  - Starting state
    - At rest in the center of road
  - Episode ends if any one of the following occurs
    - All of the tiles are visited
    - Go outside the track (-100 reward and die)

https://github.com/openai/gym/blob/master/gym/envs/box2d/car_racing.py
https://github.com/openai/gym/blob/master/gym/envs/box2d/car_dynamics.py

Car Racing

- CNN Feature Extractor
  - 이미지 데이터를 다루기 위해 합성곱 신경망(CNN) 사용



Car Racing

- CNN Feature Extractor - Torch
  - Implementation

```
class CNNFeatureExtractor(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=8, stride=4)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=4, stride=2)
        self.conv3 = nn.Conv2d(64, 64, kernel_size=3, stride=1)
        self.linear = nn.Linear(64 * 64, 128)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = F.relu(self.conv3(x))
        x = torch.flatten(x, 1)
        x = F.relu(self.linear(x))
        return x
```

입력 채널 크기 출력 채널 크기

kernel_size: kernel 크기
stride: stride 크기

Car Racing

- CNN Feature Extractor - DeZero
  - Implementation

```
class CNNFeatureExtractor(Model):
    def __init__(self):
        super().__init__()
        self.conv1 = L.Conv2d(32, kernel_size=8, stride=4)
        self.conv2 = L.Conv2d(64, kernel_size=4, stride=2)
        self.conv3 = L.Conv2d(64, kernel_size=3, stride=1)
        self.linear = L.Linear(128)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = F.relu(self.conv3(x))
        x = F.flatten(x)
        x = F.relu(self.linear(x))
        return x
```

출력 채널 크기

kernel_size: kernel 크기
stride: stride 크기

Car Racing

- Preprocessing
  - 이미지를 네트워크 입력 형식을 맞추기 위해 변환
  - Car racing의 state(observation)은 가로 96픽셀, 세로 96픽셀, 색 정보 (RGB) 3개로 이루어져 있음.
  - CNN은 RGB채널을 분리하여 사용하기 때문에 차원을 변경
    (96, 96, 3) -> (3, 96, 96)
  - 학습 안정화를 위해 이미지 픽셀 값을 0~255 -> 0~1로 정규화

```
def preprocess(state):
    # (H, W, C) -> (C, H, W)
    # 이미지 픽셀 값 정규화 0~255 -> 0~1
    return state.transpose(2, 0, 1) / 255.0
```

Car Racing - DeZero

- car_racing.py

```
pip install gym[box2d]
```

```python
import copy
from collections import deque
import random
import matplotlib.pyplot as plt
import numpy as np
import gym
from dezero import Model
from dezero import optimizers
import dezero.functions as F
import dezero.layers as L

class ReplayBuffer:
    def __init__(self, buffer_size, batch_size):
        self.buffer = deque(maxlen=buffer_size)
        self.batch_size = batch_size

    def add(self, state, action, reward, next_state, done):
        data = (state, action, reward, next_state, done)
        self.buffer.append(data)

    def __len__(self):
        return len(self.buffer)

    def get_batch(self):
        data = random.sample(self.buffer, self.batch_size)

        state = np.stack([x[0] for x in data])
        action = np.array([x[1] for x in data])
        reward = np.array([x[2] for x in data])
        next_state = np.stack([x[3] for x in data])
        done = np.array([x[4] for x in data]).astype(np.int32)
        return state, action, reward, next_state, done
```

```python
class CNNFeatureExtractor(Model):
    def __init__(self):
        super().__init__()
        self.conv1 = L.Conv2d(32, kernel_size=8, stride=4)
        self.conv2 = L.Conv2d(64, kernel_size=4, stride=2)
        self.conv3 = L.Conv2d(64, kernel_size=3, stride=1)
        self.linear = L.Linear(128)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = F.relu(self.conv3(x))
        x = F.flatten(x)
        x = F.relu(self.linear(x))
        return x


class QNet(Model):  # 신경망 클래스
    def __init__(self, action_size):
        super().__init__()
        self.feature = CNNFeatureExtractor()
        self.l1 = L.Linear(128)
        self.l2 = L.Linear(128)
        self.l3 = L.Linear(action_size)

    def forward(self, x):
        x = self.feature(x)
        x = F.relu(self.l1(x))
        x = F.relu(self.l2(x))
        x = self.l3(x)
        return x
```

Car Racing - DeZero

```python
class DQNAgent:  # 에이전트 클래스
    def __init__(self):
        self.gamma = 0.98
        self.lr = 0.0005
        self.epsilon = 0.1
        self.buffer_size = 10000  # 경험 재생 버퍼 크기
        self.batch_size = 32      # 미니배치 크기
        self.action_size = 5

        self.replay_buffer = ReplayBuffer(self.buffer_size, self.batch_size)
        self.qnet = QNet(self.action_size)         # 원본 신경망
        self.qnet_target = QNet(self.action_size)  # 목표 신경망
        self.optimizer = optimizers.Adam(self.lr)
        self.optimizer.setup(self.qnet)            # 옵티마이저에 qnet 등록

    def get_action(self, state):
        if np.random.rand() < self.epsilon:
            return np.random.choice(self.action_size)
        else:
            state = state[np.newaxis, :]  # 배치 처리용 차원 추가
            qs = self.qnet(state)
            return qs.data.argmax()

    def update(self, state, action, reward, next_state, done):
        # 경험 재생 버퍼에 경험 데이터 추가
        self.replay_buffer.add(state, action, reward, next_state, done)
        if len(self.replay_buffer) < self.batch_size:
            return  # 데이터가 미니배치 크기만큼 쌓이지 않았다면 여기서 끝

        # 미니배치 크기 이상이 쌓이면 미니배치 생성
        state, action, reward, next_state, done = self.replay_buffer.get_batch()
        qs = self.qnet(state)
        q = qs[np.arange(self.batch_size), action]

        next_qs = self.qnet_target(next_state)
        next_q = next_qs.max(axis=1)
        next_q.unchain()
        target = reward + (1 - done) * self.gamma * next_q

        loss = F.mean_squared_error(q, target)

        self.qnet.cleargrads()
        loss.backward()
        self.optimizer.update()

    def sync_qnet(self):  # 두 신경망 동기화
        self.qnet_target = copy.deepcopy(self.qnet)
```

```python
def preprocess(state):
    # (H, W, C) -> (C, H, W)
    # 이미지 픽셀 값 정규화 0~255 -> 0~1
    return state.transpose(2, 0, 1) / 255.0

episodes = 1000      # 에피소드 수
sync_interval = 10  # 신경망 동기화 주기(10번째 에피소드마다 동기화)
env = gym.make('CarRacing-v2', continuous=False, render_mode='rgb_array')
agent = DQNAgent()
reward_history = [] # 에피소드별 보상 기록

for episode in range(episodes):
    state = env.reset()[0]
    state = preprocess(state)
    done = False
    total_reward = 0

    while not done:
        action = agent.get_action(state)
        next_state, reward, terminated, truncated, info = env.step(action)
        done = terminated | truncated
        next_state = preprocess(next_state)

        agent.update(state, action, reward, next_state, done)
        state = next_state
        total_reward += reward

    # 에피소드가 끝나면 신경망 동기화

    if episode % sync_interval == 0:
        agent.sync_qnet()

    reward_history.append(total_reward)
    if episode % 10 == 0:
        print("episode :{}, total reward : {}".format(episode, total_reward))

# 에피소드별 보상 총합의 추이
plt.xlabel('Episode')
plt.ylabel('Total Reward')
plt.plot(range(len(reward_history)), reward_history)
plt.show()
```

Car Racing - DeZero

```
# 에피소드별 보상 총합의 추이
plt.xlabel('Episode')
plt.ylabel('Total Reward')
plt.plot(range(len(reward_history)), reward_history)
plt.show()

# 학습이 끝난 에이전트에 탐욕 행동을 선택하도록 하여 플레이
env2 = gym.make('CarRacing-v2', continuous=False, render_mode='human')

agent.epsilon = 0  # 탐욕 정책(무작위로 행동할 확률 ε을 0로 설정)
state = env2.reset()[0]
state = preprocess(state)

done = False
total_reward = 0

while not done:
    action = agent.get_action(state)
    next_state, reward, terminated, truncated, info = env2.step(action)
    done = terminated | truncated

    next_state = preprocess(next_state)
    state = next_state
    total_reward += reward
    env2.render()
print('Total Reward:', total_reward)
```

Car Racing - Torch

- car_racing_torch.py

```python
from collections import deque
import random
import matplotlib.pyplot as plt
import numpy as np
import gym
import torch
import torch.optim as optim
import torch.nn.functional as F
import torch.nn as nn

You, 12 minutes ago | 1 author (You)
class ReplayBuffer:
    def __init__(self, buffer_size, batch_size):
        self.buffer = deque(maxlen=buffer_size)
        self.batch_size = batch_size

    def add(self, state, action, reward, next_state, done):
        data = (state, action, reward, next_state, done)
        self.buffer.append(data)

    def __len__(self):
        return len(self.buffer)

    def get_batch(self):
        data = random.sample(self.buffer, self.batch_size)

        state = np.stack([x[0] for x in data])
        action = np.array([x[1] for x in data])
        reward = np.array([x[2] for x in data])
        next_state = np.stack([x[3] for x in data])
        done = np.array([x[4] for x in data]).astype(np.int32)
        return state, action, reward, next_state, done
```

```python
class CNNFeatureExtractor(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=8, stride=4)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=4, stride=2)
        self.conv3 = nn.Conv2d(64, 64, kernel_size=3, stride=1)
        self.linear = nn.Linear(64 * 64, 128)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = F.relu(self.conv3(x))
        x = torch.flatten(x, 1)
        x = F.relu(self.linear(x))
        return x

You, 12 minutes ago | 1 author (You)
class QNet(nn.Module):  # 신경망 클래스
    def __init__(self, action_size):
        super().__init__()
        self.feature = CNNFeatureExtractor()
        self.l1 = nn.Linear(128, 128)
        self.l2 = nn.Linear(128, 128)
        self.l3 = nn.Linear(128, action_size)

    def forward(self, x):
        x = self.feature(x)
        x = F.relu(self.l1(x))
        x = F.relu(self.l2(x))
        x = self.l3(x)
        return x
```

Car Racing - Torch

```python
class DQNAgent:  # 에이전트 클래스
    def __init__(self, device='cpu'):
        self.gamma = 0.98
        self.lr = 0.0005
        self.epsilon = 0.1
        self.buffer_size = 10000  # 경험 재생 버퍼 크기
        self.batch_size = 32      # 미니배치 크기
        self.action_size = 5
        self.device = device

        self.replay_buffer = ReplayBuffer(self.buffer_size, self.batch_size)
        self.qnet = QNet(self.action_size).to(self.device)          # 원본 신경망
        self.qnet_target = QNet(self.action_size).to(self.device)  # 목표 신경망
        self.optimizer = optim.Adam(self.qnet.parameters(), lr=self.lr)

    def get_action(self, state):
        if np.random.rand() < self.epsilon:
            return np.random.choice(self.action_size)
        else:
            state = state[np.newaxis, :]  # 배치 처리용 차원 추가
            state = torch.tensor(state, dtype=torch.float32, device=self.device)
            with torch.no_grad():
                qs = self.qnet(state)
            return qs.argmax(dim=1).item()
```

```python
    def update(self, state, action, reward, next_state, done):
        # 경험 재생 버퍼에 경험 데이터 추가
        self.replay_buffer.add(state, action, reward, next_state, done)
        if len(self.replay_buffer) < self.batch_size:
            return  # 데이터가 미니배치 크기만큼 쌓이지 않았다면 여기서 끝

        # 미니배치 크기 이상이 쌓이면 미니배치 생성
        state, action, reward, next_state, done = self.replay_buffer.get_batch()
        state = torch.tensor(state, dtype=torch.float32, device=self.device)
        action = torch.tensor(action, dtype=torch.long, device=self.device)
        reward = torch.tensor(reward, dtype=torch.float32, device=self.device)
        next_state = torch.tensor(next_state, dtype=torch.float32, device=self.device)
        done = torch.tensor(done, dtype=torch.float32, device=self.device)

        qs = self.qnet(state)
        q = qs[torch.arange(self.batch_size), action]

        with torch.no_grad():
            next_qs = self.qnet_target(next_state)
            next_q = next_qs.max(dim=1)[0]
            target = reward + (1 - done) * self.gamma * next_q

        loss = F.mse_loss(q, target)  # mean_squared_error

        self.qnet.zero_grad()  # cleargrads
        loss.backward()
        self.optimizer.step()  # update

    def sync_qnet(self):  # 두 신경망 동기화
        self.qnet_target.load_state_dict(self.qnet.state_dict())
```

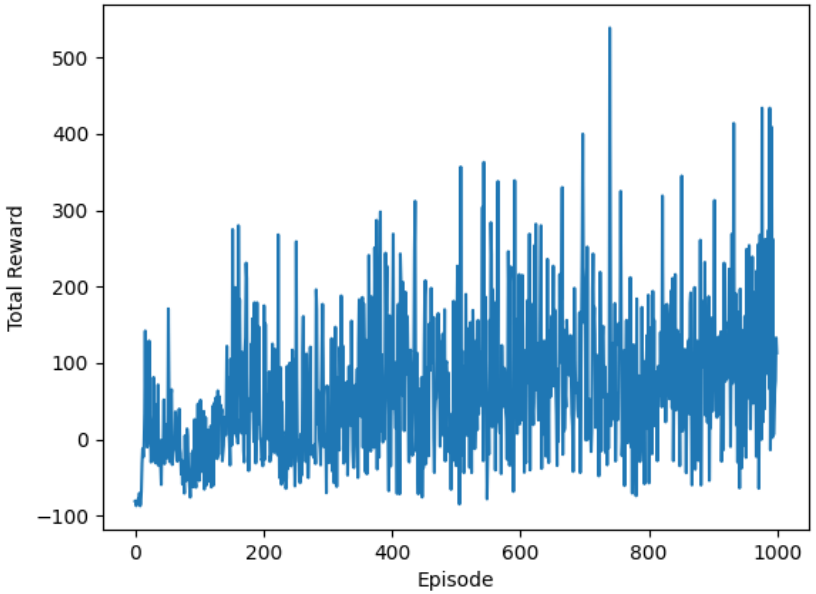
```python
def preprocess(state):
    # (H, W, C) -> (C, H, W)
    state = np.transpose(state, (2, 0, 1))
    # 이미지 픽셀 값 정규화 [0, 255] to [0, 1]
    state = state / 255.0
    return state


# Device selection
device = torch.device("cpu")
episodes = 1000      # 에피소드 수
sync_interval = 10  # 신경망 동기화 주기(10번째 에피소드마다 동기화)

env = gym.make('CarRacing-v2', continuous=False, render_mode='rgb_array')
agent = DQNAgent(device=device)
# agent.qnet.load_state_dict(torch.load('dqn_qnet_target_1000.pth'))
# agent.qnet_target.load_state_dict(torch.load('dqn_qnet_target_1000.pth'))
reward_history = [] # 에피소드별 보상 기록

for episode in range(episodes):
    state = env.reset()[0]
    state = preprocess(state)
    done = False
    total_reward = 0

    while not done:
        action = agent.get_action(state)
        next_state, reward, terminated, truncated, info = env.step(action)
        next_state = preprocess(next_state)
        done = terminated | truncated

        agent.update(state, action, reward, next_state, done)
        state = next_state
        total_reward += reward

    # 에피소드가 끝나면 신경망 동기화
    if episode % sync_interval == 0:
        agent.sync_qnet()

    reward_history.append(total_reward)
    if episode % 10 == 0:
        print("episode :{}, total reward : {}".format(episode, total_reward))

# 에피소드별 보상 총합의 추이
plt.xlabel('Episode')
plt.ylabel('Total Reward')
plt.plot(range(len(reward_history)), reward_history)
plt.show()
```

Car Racing - Torch

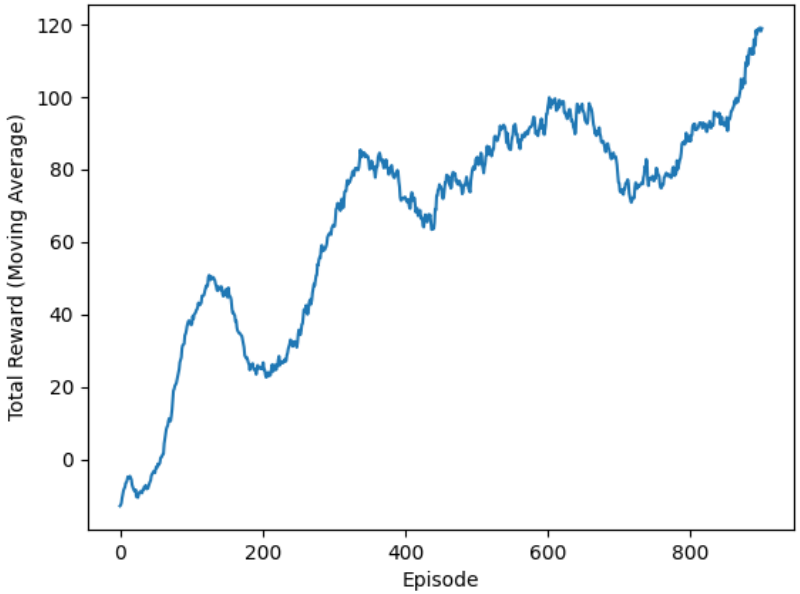
```
# 학습이 끝난 에이전트에 탐욕 행동을 선택하도록 하여 플레이
env2 = gym.make('CarRacing-v2', continuous=False, render_mode='human')

agent.epsilon = 0  # 탐욕 정책(무작위로 행동할 확률 ε을 0로 설정)
state = env.reset()[0]
state = preprocess(state)

done = False
total_reward = 0

while not done:
    action = agent.get_action(state)
    next_state, reward, terminated, truncated, info = env2.step(action)
    done = terminated | truncated

    next_state = preprocess(next_state)
    state = next_state
    total_reward += reward
    env2.render()
print('Total Reward:', total_reward)
```

Car Racing

- 학습 결과



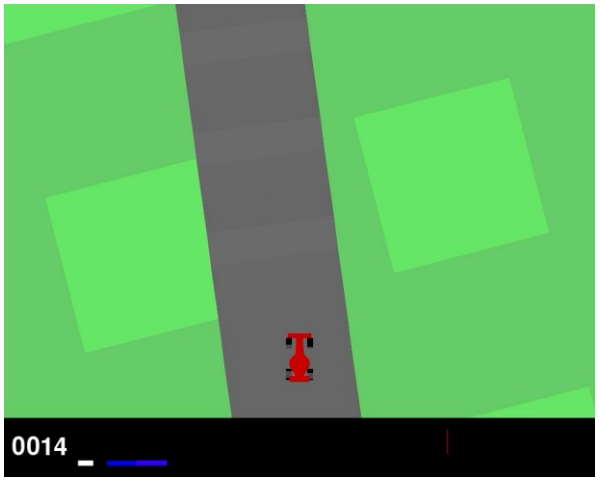
에피소드 별 보상 총합



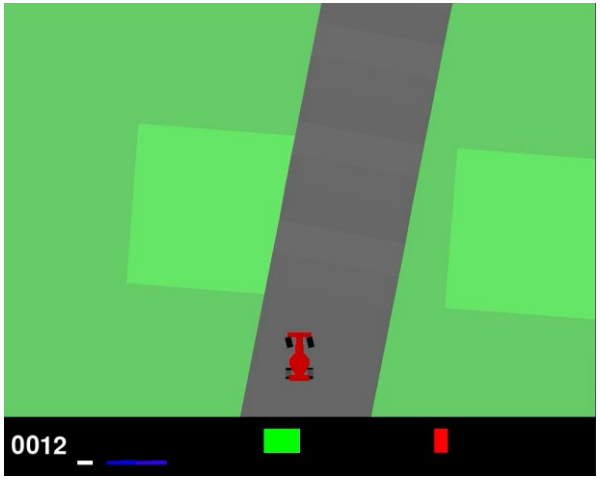
100번 평균 보상 값

Car Racing

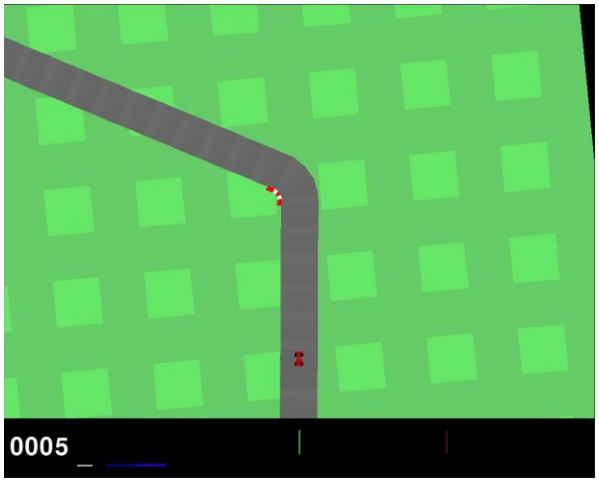
- 실행 예



500,000 step



1,000,000 step



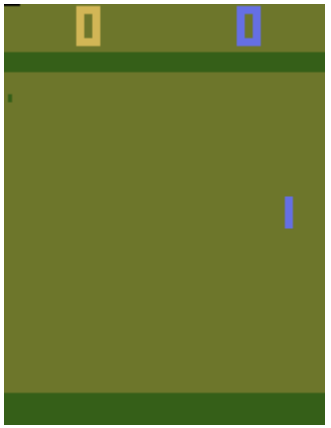
2,000,000 step

Projects #2

- Environment
  - Atari game PONG
- Method
  - DQN, Actor-Critic, A2C, PPO, SAC 등 강화학습 알고리즘 적용
  - DeZero, Torch 사용 가능
- 제출물
  - 프로그램 소스 : 재현 가능
  - PPT
    1) 강화학습 알고리즘 설명 - 핵심부분 상세히
    2) 프로그램 소스 설명
    3) 학습 결과
    4) 실행 결과 (동영상 포함)

Projects #2

- PONG Environment



Action Space	Discrete(6)
Observation Shape	(210, 160, 3)
Observation High	255
Observation Low	0
Import	`gymnasium.make("ALE/Pong-v5")`

- Action

Value	Meaning	Value	Meaning	Value	Meaning
0	NOOP	1	FIRE	2	RIGHT
3	LEFT	4	RIGHTFIRE	5	LEFTFIRE

- Reward

You get score points for getting the ball to pass the opponent's paddle. You lose points if the ball passes your paddle. For a more detailed documentation, see the AtariAge page.

https://www.gymlibrary.dev/environments/atari/pong/